

■■■

■	■
■■■■	■■■■■■■■■■■■■■■■■■■■
■■■■	Enterprise Agentic-RAG Knowledge Base
■	■■■■■Agentic AI ■■■■
■	2025303004
■	■
■	■■■■■ / ■■■■
■■■■	■
■■■■	2026 ■ 6 ■ 22 ■

■

- [■■■■■■■■■■■■■■■■■■■■](#)
- [■■Specs ■■■■](#)
- [■■■■■■■■■■■■■■■■■■■■](#)
- [■■■■■■■■■■■■■■■■■■■■](#)
- [■■■■■■■■■■](#)
- [■■■■■■■■■■■■■■■■■■■■](#)
- [■■■■■■■■■■](#)

■■■■■■■■■■■■■■■■■■■■

ID		
US-01		P0
US-02		P0
US-03		P0
US-04		P1
US-05		P2

2.2 Architecture Spec



6

users		id, username, email, password_hash
documents		id, user_id, filename, file_type, status
document_chunks		id, document_id, chunk_index, content

conversations	■■■■	id, user_id, title
messages	■■■■	id, conversation_id, role, content, citations_json
retrieval_logs	■■■■■	id, message_id, query, plan, retrieved_docs

■■■■

■■■■	■■■■	TTL
■■■■■	Redis List	24h
■■■■■	Redis String	5min
■■■■■	Redis Hash	7d

2.3 API Spec

■■■■

Method	Path	■■
POST	/api/auth/register	■■■■
POST	/api/auth/login	■■■■
POST	/api/documents/upload	■■■■
POST	/api/documents/{id}/process	■■■■
GET	/api/documents/	■■■■
POST	/api/chat	■■■■
POST	/api/chat/stream	SSE ■■■■
GET	/api/conversations/	■■■■
GET	/api/conversations/{id}	■■■■
GET	/api/conversations/messages/{id}	■■■■
GET	/api/retrieval/debug/{id}	■■■■

3.3



- PDF (PyMuPDF)DOCX (python-docx)TXTMarkdown
- RecursiveCharacterTextSplitterchunk_size=500overlap=100
- EmbeddingBGE Large Zh (bge-large-zh-v1.5)1024



- Vector Search (Milvus) + BM25RRF
- BGE Reranker (bge-reranker-large)
- 2

3.4

Agent	LangGraph	
	Milvus	
	Redis	
Embedding	BGE Large Zh	MTEB
Reranker	BGE Reranker	Embedding
	FastAPI	
	Vue3	API



4.1 LangGraph Workflow


```

# backend/src/workflows/rag_workflow.py def build_rag_workflow() -> StateGraph: """Build the RAG
workflow graph with all agents.""" graph = StateGraph(RAGState) # Add agent nodes
graph.add_node("planner", planner_agent) graph.add_node("query_agent", query_agent)
graph.add_node("retriever", retriever_agent) graph.add_node("rerank", rerank_agent)
graph.add_node("reflection", reflection_agent) graph.add_node("answer", answer_agent) # Define
workflow edges graph.set_entry_point("planner") graph.add_edge("planner", "query_agent")
graph.add_edge("query_agent", "retriever") graph.add_edge("retriever", "rerank")
graph.add_edge("rerank", "reflection") # Conditional edge: retry or answer
graph.add_conditional_edges("reflection", should_retry, {"answer": "answer", "query_agent":
"query_agent"}) graph.add_edge("answer", END) return graph.compile()

```

4.2 RAGState

```

# backend/src/agents/state.py class RAGState(TypedDict): original_query: str rewritten_query:
str expanded_queries: list[str] retrieval_plan: str retrieved_documents: list[RetrievalDocument]
reranked_documents: list[RetrievalDocument] reflection_result: str reflection_passed: bool
query_count: int final_answer: str chat_history: list[ChatHistory] chat_history_context: list[str]
chat_messages: list[ChatMessage] chat_id: str | None message_id: str | None workflow_trace: list[dict]

```

4.3 Planner Agent

```

# backend/src/agents/planner.py async def planner_agent(state: RAGState) -> dict: """Analyze
user query and generate retrieval plan.""" query = state["original query"] chat_history =
state.get("chat_history", []) # Build conversation history context history_text = "" if
chat_history: history_text = "\n".join([f"{msg.type}: {msg.content}" for msg in
chat_history][-6:[]]) llm = get_llm() prompt = f"""##### {query}
{history_text}""" if history_text else "" """JSON""" """ {} "needs_decomposition":
true/false, "sub_queries": [{"1", "2"}, "strategy": "strategy", "key_entities":
["1", "2"]} """ """JSON""" """ try: response = await llm.ainvoke(prompt)
plan_text = _extract_text(response.content) plan = json.loads(plan_text) except Exception as e:
logger.warning("planner_parse failed", error=str(e)) plan = {"needs_decomposition": False,
"sub_queries": [query]} return {"retrieval_plan": json.dumps(plan, ensure_ascii=False),
"workflow_trace": state.get("workflow_trace", []) + [{"step": "planner", "output": plan}],

```

4.4

```

# backend/src/retrieval/hybrid.py async def hybrid_search(query: str, top_k: int = 10) ->
list[dict]: """Vector BM25""" # Check cache first cache_key =
"search:{hashlib.md5(query.encode()).hexdigest():{top_k}" try: redis = await get_redis()
cached = await redis.get(cache_key) if cached: return json.loads(cached) except Exception: pass
# Run BM25 and vector search in parallel bm25_index = get_bm25_index() bm25_task =
asyncio.to_thread(bm25_index.search, query, top_k) vector_task = search_similar(query,
top_k=top_k) bm25_results, vector_results = await asyncio.gather(bm25_task, vector_task)
return_exceptions=True, ) # Merge by chunk id merged: dict[int, dict] = {} for r in
bm25_results: key = r["chunk_id"] merged[key] = {"bm25 score": r["score"], "vector score":
0.0} for r in vector_results: key = r["chunk_id"] if key in merged: merged[key]["vector score"]
= r["score"] else: merged[key] = {"bm25 score": 0.0, "vector score": r["score"]} # RRF
Ranking ranked = rrf_rank(list(merged.values()), top_k=top_k) # Cache results try: redis = await
get_redis() await redis.setex(cache_key, 300, json.dumps(ranked, ensure_ascii=False)) except
Exception: pass return ranked def rrf_rank(results: list[dict], k: int = 60, top_k: int = 10) ->
list[dict]: """Reciprocal Rank Fusion""" for r in results: bm25_rank = r.get("bm25 rank", k + 1)
vector_rank = r.get("vector rank", k + 1) r["rrf score"] = 1.0 / (k + bm25_rank) + 1.0 / (k
+ vector_rank) return sorted(results, key=lambda x: x["rrf score"], reverse=True)[:top_k]

```

4.5 SSE

```
backend/src/api/v1/chat.py @router.post("/stream") async def chat_stream_endpoint( request: ChatRequest, current_user: User = Depends(get_current_user), ): """SSE streaming with real-time agent progress events.""" async def event_generator(): try: async for event_type, event_data in chat_stream( user_id=current_user.id, message=request.message, conversation_id=request.conversation_id, ): if event_type == "progress": yield { "event": "progress", "data": json.dumps(event_data, ensure_ascii=False), } elif event_type == "final": yield { "event": "token", "data": json.dumps({"text": event_data.get("answer", "")}, ensure_ascii=False), } yield { "event": "final", "data": json.dumps(event_data, ensure_ascii=False, default=str), } elif event_type == "error": yield { "event": "error", "data": json.dumps(event_data, ensure_ascii=False), } except Exception as e: yield { "event": "error", "data": json.dumps({"detail": str(e)}) } return EventSourceResponse(event_generator())
```

4.6 AI IDE

Claude Code AI IDE

- Claude Code 1. Claude Code LangGraph Workflow 2. Claude Code Agent 3. Claude Code

AI IDE

	AI IDE
	Agent
	LangGraph WorkflowAgentAPI
	Mock
	SpecsAPI

5.1

Input	Processing	Latency	Output
Document	3 LLM	~60-70s	LLM API 15-25s
Document	PDF + Embedding	~100/min	Embedding CPU
Document	BM25 + Vector + RRF	< 500ms	Redis < 50ms
Rerank	BGE Reranker	~10s	< 200ms

LLM - BM25 + Vector asyncio.gather - Redis
5min TTL - Reranker

5.4 Demo

1280x720

1

Agent

→ → →

2

→ PDF → →

3

Agent Rerank

→ → →

4 SSE

Agent

→ Agent

6.1

- 1. Agent Agent
- 2. Knowledge Graph
- 3. Milvus Redis

6.2

Phase 2		RBAC + JWT
Phase 2		+
Phase 3		+
Phase 3		Neo4j + GraphRAG

6.3 AI

Agent	Phase 2	Agent	Agent	Phase 3

7.1

Agent

SDD Spec



AI

7.2 ■■■■■

1. **Agent**  **Agent** 
2. 
3. **LangGraph** 

7.3 ■■

[illegible]

1. LangGraph Documentation. <https://python.langchain.com/docs/langgraph>
2. Lewis, P., et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." NeurIPS 2020.
3. BGE Embedding Models. <https://huggingface.co/BAAI/bge-large-zh-v1.5>
4. Milvus Documentation. <https://milvus.io/docs>
5. FastAPI Documentation. <https://fastapi.tiangolo.com/>



A. ☐ ☐ ☐ ☐

GitHub: <https://github.com/wseacat/cs599-project.git>

B. ■■■■■■

```
export ENV_EXAMPLE_APP_ENV=development SECRET_KEY=your-secret-key LLM_API_KEY=your-api-key  
LLM_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1 LLM_MODEL=gwen-plus  
DATABASE_URL=mysql+aiomysql://root:changeme@localhost:3306/enterprise_rag REDIS_HOST=localhost  
REDIS_PORT=6379 MILVUS_HOST=localhost MILVUS_PORT=19530
```

C. ■■■■

```
# 1. cp .env.example .env # 2. docker compose up -d # 3. cd backend npm
install --a uvicorn src main:app --reload --port 8000 # 4. cd frontend npm install npm
run dev
```